

CodeSheriff: Design and Development of an Agentic Security Auditor for an SFI-Enabled Confidential CI/CD Execution Layer Using Bayesian Vulnerability Inference

Shubhankar Vishnu Patil

Dept. of Information Technology
G H Raisoni College of Engineering and Management
Pune, India
shubhankarp258@gmail.com

Yash Suresh Kodgirwar

Dept. of Information Technology
G H Raisoni College of Engineering and Management
Pune, India
yashkodgirwar@gmail.com

Kunal Kundu

Dept. of Information Technology
G H Raisoni College of Engineering and Management
Pune, India

Vivek Dnyaneshwar Joshi

Dept. of Information Technology
G H Raisoni College of Engineering and Management
Pune, India
vivekdjoshi05@gmail.com

Nirmalya Mukhopadhyay

Dept. of Information Technology
G H Raisoni College of Engineering and Management
Pune, India

Abstract—Modern software development increasingly relies on Continuous Integration/Continuous Deployment (CI/CD) pipelines to deliver software rapidly, but automated security review at the pull request stage remains limited. Existing security tools rely on rigid static analysis or homogeneous multi-agent approaches, which may generate false positives or overlook complex vulnerabilities. This project will present CodeSheriff, an agentic security auditor that will analyze GitHub pull requests using structural, semantic, contextual, and SFI-based confidential execution analysis. A Bayesian inference engine will fuse evidence from multiple agents to generate confidence-aware vulnerability assessments, while an automated patch generation and verification module will recommend and validate fixes through GitHub and a web dashboard. The system will enhance CI/CD security by delivering reliable, explainable, and statistically grounded vulnerability detection while reducing manual review effort.

Index Terms—vulnerability detection, multi-agent systems, software security, continuous integration and continuous deployment

I. INTRODUCTION

In today's software industry, developers frequently push code to the main branch through Pull Requests (PRs). However, security analysis of each Pull Request is still largely performed manually, making the review process more time-consuming. Teams either use basic approaches that only catch known, simple mistakes, or they don't have enough time to properly review every pull request. This is a big problem because even one missed vulnerability can put the whole application at risk once the code is merged and deployed. On top of this, attackers are now also targeting the Continuous

Integration/Continuous Deployment (CI/CD) that's builds and runs the code.

Solutions that exist today don't fully solve this problem. Traditional static analysis approaches identify fixed patterns but are unable to catch tricky ones, so they generate a lot of false alarms. Some recent tools use multiple artificial intelligence (AI) agents, but all these agents basically read the same code in the same way, so they don't really give different opinions. When these approaches combine results, they just use simple voting, which doesn't tell you how confident the final answer actually is.

In this paper, we are proposing CodeSheriff to fix this problem. It uses four different types of agents that look at the code from different perspectives. Static agent checks the code structure, the meaning of the code is understood by the semantic agent, the context agent checks whether accepting this PR could bring back any vulnerability from previously accepted PR, and the runtime agent actually watches the code run inside a safe, isolated environment using software fault isolation (SFI). All this information is then combined using a Bayesian model, which gives a clear probability of whether the code is vulnerable, along with how confident the system is. This same safe environment also protects the CI/CD workflow, since it only releases secret keys when it is sure the correct, verified code is running.

The rest of this paper is structured as follows. Section II talks about related work already done in this area. Section III explains our proposed method, its algorithms and its implementation. Section IV describes the experimental evaluation, Section V discusses the results, and Section VI concludes the

paper.

II. LITERATURE SURVEY

Early learning-based studies explored structural and sequential representations for vulnerability detection. Li and Yang [1] compared sequence and graph-based deep learning methods, identifying challenges in capturing long-range dependencies and generalizing to noisy data. CODE-SMASH [2] and XTransformer [3] employed hierarchical neural and transformer architectures to capture multi-level code representations and improve similarity-based detection, while remaining sensitive to computational cost and dataset characteristics. Abbasi and Shameli-Sendi [4] used Code Property Graph metrics with machine learning for interpretable vulnerability and vulnerability-type prediction, but their approach was limited to C/C++. DeepSecure [5] combined CodeBERT with bidirectional long short-term memory (BiLSTM) and convolutional neural network (CNN) layers for python vulnerability detection, incorporating robustness and confidence calibration, but remained language-specific and affected by noisy labels.

Transformer and large language model (LLM) based approaches have subsequently improved semantic analysis of source code. Shestov *et al.* [6] fine-tuned WizardCoder using parameter-efficient learning for vulnerability classification, but remained limited to function-level analysis. Tamberg and Bahsi and Gnieciak and Szandala [7], [8] benchmarked LLMs against vulnerability detection tasks and traditional static analysis tools, showing strong potential but persistent false positives, hallucinations, localization errors, and inference costs. Gülmez [9] surveyed code-generation LLMs and highlighted challenges in structural correctness, repository-level reasoning, and security. Chen *et al.* [10] and Su *et al.* [11] similarly identified limitations involving context windows, hallucination, dataset quality, generalization, and interpretability. Germano *et al.* [12] reviewed LLM-based vulnerability detection, repair, and explanation, noting that research remains more focused on detection than automated repair and explanation.

To address the limitations of isolated code analysis, recent studies have incorporated contextual and repository-level information. Antal *et al.* [13] evaluated retrieval augmented generation (RAG) for real-world Java vulnerability detection using retrieved vulnerable and secure examples, but observed false positives and retrieval sparsity. AegisGuard [14] incorporated contextual information for semantic vulnerability detection and risk stratification, while SCOUT [15] enabled interactive repository exploration to retrieve relevant code context. Göçmen *et al.* [16] integrated repository history and dependency analysis into pull-request reviews for change-impact estimation, although static dependency analysis cannot fully capture dynamic behavior.

Multi-agent approaches have further introduced collaborative security reasoning. Wei *et al.* [17] used specialized roles and collaborative reasoning for vulnerability auditing, while MAVUL [18] combined vulnerability analysis, security architecture review, and iterative evaluation. Reddy *et al.* [19] explored knowledge-based agentic AI for automated vulnerabil-

ity triage, and Syed *et al.* [20] investigated an observe–reason–act–verify framework for autonomous software-supply-chain defense. These studies demonstrate the potential of agentic security workflows, while also highlighting challenges such as reduced performance on complex cases, token overhead, and the need for human intervention.

However, existing approaches do not effectively combine structural, semantic and contextual analysis within a unified vulnerability detection framework. We integrate these approaches through specialized agents and confidence-aware vulnerability inference, while providing explanations and patch recommendations within the GitHub pull-request workflow.

III. PROPOSED METHODOLOGY

A vulnerability can hide in the structure of the code, in its intent, in its departure from the repository’s own habits, or only in what the code does when it runs. To cover these four views, CodeSheriff uses four independent agents: a Structural agent for data flow, a Semantic agent for intent, a Context agent for repository precedent, and a Runtime agent for observed execution. No agent sees the output of another, so their errors stay independent. A Bayesian inference engine combines their statements into one *calibrated* probability: a change reported at 87% should be vulnerable about 87% of the time [21].

A. Mathematical Model

CodeSheriff evaluates a pull request through six stages: change extraction, structural, semantic, context and runtime analysis, and Bayesian fusion with the alert decision.

1) *Change Extraction*: A diff fragment is too small to reason about and a whole file mixes unrelated code, so the unit of analysis is one changed function. Let B_f and H_f be the base and head versions of a changed file f . The changed lines and the set of units are

$$L_f = \text{Diff}(B_f, H_f), \quad U = \{u = \text{Outer}(f, \ell) : \ell \in L_f\} \quad (1)$$

where $\text{Outer}(f, \ell)$ is the outermost function of H_f that contains line ℓ . Each unit u holds the file, the function name, its old and new source, and its changed lines. Lines outside any function form one module-level unit. A unit too large to analyse is never truncated; the agents abstain on it.

2) *Structural Agent*: The Structural agent follows data through a definition–use graph from each function parameter (treated as untrusted) to a dangerous call, or *sink*, such as `cursor.execute` or `os.system`. When an unsanitised path reaches a sink, the structural score is

$$S_{\text{str}} = \text{clip}_{[0,1]}(0.45 + \delta + 0.15\nu - 0.25\rho - 0.20\theta - \lambda) \quad (2)$$

where δ is 0.20 for a critical sink and 0.10 for a high one, $\nu = 1$ when the code is network-facing, $\rho = 1$ when the path passes a partial sanitizer, $\theta = 1$ in a test file, and $\lambda \leq 0.05$ is a small penalty for long paths. A second backend, Semgrep [22], scores the same unit, and the witness keeps the larger of the two scores, because both read the same source text.

3) *Semantic Agent*: Some weaknesses, such as a missing authorization check, have no fixed syntax. The Semantic agent asks a language model for the intent of the function, its trust boundaries and the security rule the change breaks. The model is sampled $n = 3$ times, and the score for weakness class c is the share of samples that report it:

$$S_{\text{sem}}(c) = \frac{m_c}{n} \quad (3)$$

where m_c is the number of accepted samples reporting c . A sample is accepted only when it passes the hallucination gate

$$G(\phi) = [c \in \mathcal{C}] \wedge [\text{file matches}] \wedge [\text{sink} \subseteq \text{src}] \wedge [\text{lines} \subseteq u] \quad (4)$$

where $\mathcal{C}$ is the closed set of ten in-scope CWEs [23], the quoted sink must appear word-for-word in the source, and the reported lines must lie inside the unit. The code is wrapped between random markers created for each request, so text inside it cannot act as an instruction.

4) *Context Agent*: A change can be unsafe only because the repository normally does something else, for example when every similar endpoint checks ownership except the new one. The Context agent retrieves similar merged functions of the same repository (384-dimensional local embeddings in pgvector) and reports a security control γ that they apply and the unit does not:

$$S_{\text{ctx}}(\gamma) = b \cdot \min\left(1, \frac{\bar{s}}{0.85}\right) \quad (5)$$

$$b = \begin{cases} 0.85, & \text{same function, or } k \geq 3 \\ 0.65, & k = 2 \end{cases}$$

where $\bar{s}$ is the mean similarity of the supporting functions and k is the number of sibling functions that apply γ . A single sibling ($k = 1$) is treated as coincidence and reports nothing. The agent reports only authorization weaknesses (CWE-862 and CWE-639).

5) *Runtime Agent*: The Runtime agent executes the changed function once inside a WebAssembly [24] sandbox (Wasmtime with WASI), which provides software fault isolation [25]: there is no network, no environment and no host file, and fuel, time and memory are capped. Each parameter i receives a unique random token t_i . A value v is tainted, and a detection is reported, when

$$\text{tainted}(v) \iff \exists i : t_i \subseteq v, \quad S_{\text{rt}} = 0.85 \quad (6)$$

where v is the argument that reaches a sink. A direct observation needs no graded score, so S_{rt} is a constant. If the value reached the sink only because the sandbox had to answer a check it could not evaluate, the agent abstains instead.

6) *Bayesian Fusion and Decision*: Every agent a returns one of three statements: a *detection* with a score S_a , a *silence* (it ran and found nothing), or an *abstention* (it could not run). The statement is placed in a cell

$$z_a = \begin{cases} \text{HIGH}, & S_a \geq 0.8 \\ \text{MEDIUM}, & 0.5 \leq S_a < 0.8 \\ \text{LOW}, & S_a < 0.5 \\ \text{SILENCE}, & \text{ran, found nothing} \end{cases} \quad (7)$$

Algorithm 1: Evidence Generation for One Unit

Input: Changed function u
Output: Statements $E = \{e_{\text{str}}, e_{\text{sem}}, e_{\text{ctx}}, e_{\text{rt}}\}$

```

1 foreach agent  $a$  in  $\{\text{Structural}, \text{Semantic}, \text{Context}, \text{Runtime}\}$ 
  do
2   if  $a$  cannot run on  $u$  (timeout, no tool, too large) then
3      $e_a \leftarrow \text{Abstention}(\text{reason});$ 
4   else
5      $F \leftarrow \text{findings of } a \text{ on } u;$  // EQS. (2) -- (6)
6      $e_a \leftarrow \text{Detection}(S_a, c)$  if  $F \neq \emptyset$  else Silence;
7 return  $E;$ 
```

Each cell has a likelihood ratio (LR) fitted on labelled data:

$$\Lambda_a(z) = \frac{P(z | V)}{P(z | \neg V)} \approx \frac{(k_V + 1)/(N_V + 4)}{(k_S + 1)/(N_S + 4)} \quad (8)$$

where k_V and k_S count vulnerable and safe examples that fell in cell z , and N_V and N_S are the totals. Adding one (Laplace smoothing) avoids division by zero. An abstention, or a cell never observed, has $\Lambda = 1$, so an agent that could not look never changes the result. Each LR is clipped to $[0.05, 20]$. Starting from the prior π , the posterior probability of a vulnerability V given the evidence E is

$$P(V | E) = \frac{O}{1 + O}, \quad O = \frac{\pi}{1 - \pi} \prod_{a=1}^4 \Lambda_a(z_a) \quad (9)$$

where O is the posterior odds. The product always has four factors, one per agent, and silences contribute factors below 1, so the probability can fall as well as rise. The finding is finally labelled

$$\text{Verdict} = \begin{cases} \text{Alert}, & P(V | E) \geq \tau \\ \text{No alert}, & \text{otherwise} \end{cases} \quad (10)$$

where $\tau = 0.227$ is selected on the validation split.

Worked example. With $\pi = 0.03$ the prior odds are 0.031. A medium structural detection ($\Lambda = 15.56$), a high semantic detection (9.48), a context abstention (1) and a high runtime detection (6.92) give $O \approx 31.6$ and $P(V | E) \approx 0.97$. A high semantic detection alone, with the other three agents abstaining, gives exactly 0.227: the threshold means that one strong finding nobody contradicts is just enough to alert.

B. Algorithms

The framework is implemented as three algorithmic stages: evidence generation by the agents, Bayesian fusion, and patch verification.

Algorithm 1 runs the four agents independently on one unit. Every agent always returns a statement, so a failure is recorded as an abstention and is never confused with “found nothing”.

Algorithm 2 multiplies the prior odds by one fitted ratio per agent and converts the result into a probability, which is compared with the threshold.

Algorithm 3 drafts up to three repairs. Each rung ends as passed, failed or *not run*, so a check that could not run is

Algorithm 2: Bayesian Fusion and Alert Decision

Input: Statements E , fitted ratios Λ , prior π , threshold τ
Output: Posterior $P(V | E)$, Verdict

```

1  $O \leftarrow \pi / (1 - \pi)$ ;
2 foreach agent  $a$  do
3    $z_a \leftarrow \text{cell of } e_a$ ; // Eq. (7)
4    $O \leftarrow O \times \Lambda_a(z_a)$ ; //  $\Lambda = 1$  if abstained
5  $P(V | E) \leftarrow O / (1 + O)$ ;
6 Verdict  $\leftarrow$  Alert if  $P(V | E) \geq \tau$  else No alert;
7 return  $P(V | E)$ , Verdict;
```

Algorithm 3: Patch Drafting and Verification

Input: Alerted finding on unit u

Output: Suggested change, or a recorded reason

```

1  $R \leftarrow \emptyset$ ; // rejection reasons
2 for  $i = 1$  to 3 do
3    $p \leftarrow \text{Draft}(u, R)$ ; // never sees rejected drafts
4   run rungs: parses, keeps signature, names resolve,
   changes code, weakness gone, no new weakness;
5   if all rungs passed then return  $\text{Suggest}(p)$  if  $p$  lies in
   the diff;
6    $R \leftarrow R \cup \{\text{reasons of failed rungs}\}$ ;
7 return no suggestion, with reasons;
```

never shown as passed. The structural and runtime agents re-examine the repaired function. A verified repair is posted as a GitHub suggested change only if it lies inside the changed lines; CodeSheriff never commits or merges.

C. Technical Workflow

The CodeSheriff workflow (Fig. 1) consists of five stages:

- 1) **Trigger:** opening or updating a pull request sends a webhook. The API verifies its HMAC-SHA256 signature, records an audit, queues it and replies within three seconds.
- 2) **Extraction:** a worker downloads the base and head files and builds one unit per changed function using (1).
- 3) **Agent analysis:** the four agents analyse every unit independently (Algorithm 1).
- 4) **Fusion and decision:** the engine computes $P(V | E)$ and the verdict (Algorithm 2); alerted findings go to the patch stage (Algorithm 3).
- 5) **Reporting:** one comment summarises the audit on the pull request, and every statement, probability and per-agent factor is stored for the dashboard.

D. Implementation

CodeSheriff is a Python 3.12 monorepo. The API and the worker run as separate processes with separate credentials: the API holds the webhook secret, the worker holds the model key, and a queued task carries only an audit identifier. Source code is kept in memory only; the database stores findings and hashes. The main components are:

- **Backend:** FastAPI, Celery and Redis.

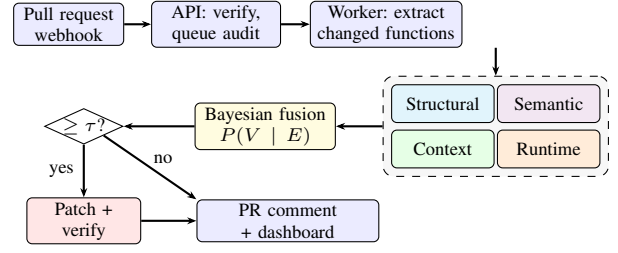


Fig. 1. Workflow of CodeSheriff. The four agents run blind.

- **Database:** PostgreSQL 16 with pgvector, SQLAlchemy, Alembic.
- **Agents:** tree-sitter and networkx taint engine with Semgrep; Gemini Flash-Lite ($n = 3$); bge-small-en-v1.5 embeddings; Wasmtime with a WASI build of CPython.
- **Dashboard:** Next.js, TypeScript and Recharts (read-only; the threshold cannot be changed per repository).
- **Quality:** 1249 tests, strict type checking, and import rules that stop any agent from importing another agent or the database.

IV. EXPERIMENTAL EVALUATION**A. Experimental Setup**

The ground truth is a hand-written corpus of 76 Python functions in 38 *twin pairs*: the same function once vulnerable and once fixed, over ten CWEs (22, 78, 79, 89, 94, 502, 639, 798, 862, 918). Pairs are split by pair into calibration (46 units), validation (14) and test (16). Ratios are fitted only on calibration, the threshold is chosen only on validation, and the test split is reserved for a single final run. No agent can read a label. Because every case has a twin, the corpus is half vulnerable, so the prior is *declared* as $\pi = 0.03$ and all metrics are weighted so that vulnerable claims carry 3% of the weight. The experiments ran on an Intel Core i5 machine with 16 GB RAM using recorded model responses, so every number is exactly reproducible.

Detection is measured by precision (P), recall (R) and F_1 . Calibration is measured by the Brier score [26] and the expected calibration error (ECE) [21]:

$$\text{Brier} = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2, \quad \text{ECE} = \sum_{b=1}^{10} \frac{|B_b|}{N} |\text{freq}_b - \text{conf}_b| \quad (11)$$

where p_i is the predicted probability, $y_i \in \{0, 1\}$ the true label, and B_b the claims in probability bin b . Lower is better for both.

B. Individual Agents and Fitted Ratios

Measured alone on the calibration split, the Structural agent detected 13/13 cases in its scope, the Context agent 5/5 and the Runtime agent 9/9, each with zero false positives on the safe twins. The Semantic agent had no hallucinated sinks, no successful prompt injection, and correctly passed 94% of safe twins. Table I gives the fitted ratios. A medium structural

TABLE I
FITTED LIKELIHOOD RATIOS (– : NEVER OBSERVED, LR = 1)

Agent	High	Medium	Low	Silence
Structural	2.22	15.56	–	0.065
Semantic	9.48	0.56	0.74	0.304
Context	2.00	4.00	2.00	0.167
Runtime	6.92	–	–	0.115

TABLE II
COMPARISON USING IDENTICAL AGENT STATEMENTS (BEST IN BOLD)

Split	System	ECE↓	Brier↓	P↑	R↑	F_1 ↑
Valid.	Prior only	0.000	0.029	0.000	0.000	0.000
	Majority vote	0.141	0.059	0.076	1.000	0.142
	CodeSheriff	0.027	0.013	1.000	0.857	0.923
Calib.	Prior only	0.000	0.029	0.000	0.000	0.000
	Majority vote	0.092	0.068	0.161	0.957	0.276
	CodeSheriff	0.009	0.010	1.000	0.696	0.821

detection makes a vulnerability $15.6\times$ more likely, while structural silence makes it about $15\times$ less likely. A low or medium semantic detection (one or two of three samples) has an LR below 1: such findings were more often safe. Voting would count them as “yes”; the fitted ratio correctly treats them as weak evidence against.

C. Comparative Analysis

CodeSheriff is compared with two baselines built on the *same* agent statements, so only the combination rule differs: *majority vote* (share of agents that detected, abstentions ignored) and *prior only* (no agent). All use $\tau = 0.227$. Table II gives the results.

D. Ablation Analysis

Each agent is removed in turn and treated as an abstention (factor 1). Fig. 2 shows F_1 on both splits.

E. Assumptions and Limitations

The corpus is small, author-written, Python-only and limited to ten CWEs. The validation split has only 15 claims and the threshold was chosen on it, so its figures are selection-time estimates; the test split is not yet evaluated. Semgrep has no Windows build, so the structural ratios describe the taint engine alone. The 3% base rate is declared, not measured.

V. RESULTS AND DISCUSSION

On the validation split CodeSheriff reached $F_1 = 0.923$ with perfect precision, alerting on 6 of 15 claims (Table II). Majority vote over the same statements found every vulnerability, but most of its alerts were false ($F_1 = 0.142$) because it has no prior to hold one agent’s detection back, and it was five times worse calibrated (ECE 0.141 against 0.027). The prior-only baseline has a perfect ECE only because it detects nothing; its Brier score (0.029 against 0.013) exposes this. The same ordering holds on the calibration split.

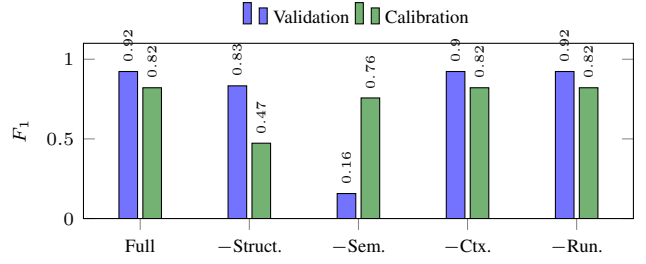


Fig. 2. Leave-one-agent-out ablation (F_1 at $\tau = 0.227$).

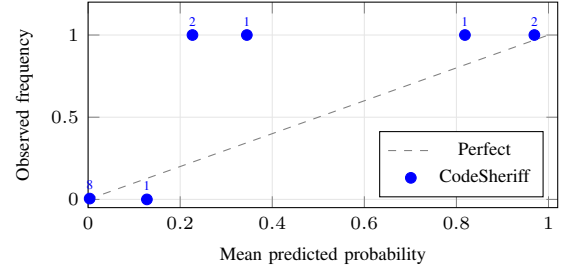


Fig. 3. Reliability diagram, validation split (labels: claims per bin).

Fig. 3 is the reliability diagram. Most claims receive a very low probability and are indeed safe, and all high-probability claims are vulnerable. The two middle bins lie above the diagonal, meaning the system was slightly *under*-confident there, but they hold only one or two claims each and carry little weight.

The ablation (Fig. 2) shows that the two splits disagree about which agent matters most. On validation, removing the Semantic agent drops F_1 from 0.923 to 0.157; on calibration, removing the Structural agent drops it from 0.821 to 0.473. No single agent is enough on both splits, which supports using heterogeneous agents. Removing the Context or Runtime agent leaves every decision unchanged; this is a null result, not redundancy, since both have narrow scopes and mostly confirm decisions another agent already made.

VI. CONCLUSION

This paper presented CodeSheriff, a pull-request security auditor in which four agents built on different techniques analyse each changed function independently, and a Bayesian engine turns their statements into a calibrated probability. The design separates “found nothing” from “could not look”, and fits every ratio and the threshold on labelled data. On the validation split it reached $F_1 = 0.923$ with perfect precision and ECE 0.027, while majority voting over identical statements reached only 0.142. Unlike related systems [5], [14], [18], [19], it pairs verified patch suggestions with a probability whose reliability is itself measured.

VII. FUTURE SCOPE

The test split will be evaluated once on a Linux image where Semgrep is active, and Semgrep, Bandit and a single-pass LLM will be scored under the same protocol. The corpus

will be enlarged with real vulnerability-fixing commits, and support extended to JavaScript and TypeScript.

ACKNOWLEDGMENT

The authors thank the Department of Information Technology, G H Raisoni College of Engineering and Management, Pune, for its resources, and the project guides for their support.

REFERENCES

- [1] G. Li and Y. Yang, "On the code vulnerability detection based on deep learning: A comparative study," *IEEE Access*, 2024.
- [2] S. Han, H. Nam, J. Kang, K. Kim, S. Cho, and S. Lee, "Code-smash: Source-code vulnerability detection using siamese and multi-level neural architecture," *IEEE Access*, 2024.
- [3] S. Han, M. Kim, J. Kang, K. Kim, S. Lee, and S. Lee, "Similarity-based source code vulnerability detection leveraging transformer architecture: Harnessing cross-attention for hierarchical analysis," *IEEE Access*, 2024.
- [4] B. K. E. Abbasi and A. Shameli-Sendi, "Leveraging key source code metrics for predicting software vulnerabilities with machine learning," *IEEE Access*, 2026.
- [5] R. A. Almutairi and N. Abbadeni, "Deepsecure: Hybrid deep learning framework for detecting security vulnerabilities in python code," *IEEE Access*, 2026.
- [6] A. Shestov, R. Levichev, R. Mussabayev, E. Maslov, P. Zadorozhny, A. Cheshkov, R. Mussabayev, A. Toleu, G. Tolegen, and A. Krassovitskiy, "Finetuning large language models for vulnerability detection," *IEEE Access*, 2025.
- [7] K. Tamberg and H. Bahsi, "Harnessing large language models for software vulnerability detection: A comprehensive benchmarking study," *IEEE Access*, 2025.
- [8] D. Gnieciak and T. Szandala, "Large language models versus static code analysis tools: A systematic benchmark for vulnerability detection," *IEEE Access*, 2025.
- [9] B. Gülmez, "Code generation with large language models: a survey from neural program synthesis to autonomous software development," *Applied Intelligence*, vol. 56, no. 6, p. 200, 2026.
- [10] Y. Chen, Y. Shen, T. Wang, S. Ou, R. Wang, Y. Li, and Z. Pan, "Software defect detection using large language models: a literature review," *Frontiers of Computer Science*, vol. 20, no. 6, p. 2006202, 2026.
- [11] H. Su, Z. Xu, Y. Zhang, and Q. Tan, "Source code vulnerability detection based on deep learning: a review," *Cybersecurity*, vol. 9, no. 1, p. 2, 2026.
- [12] L. B. Germano, R. R. Goldschmidt, R. C. Noya, and J. C. Duarte, "A systematic review on detection, repair, and explanation of vulnerabilities in source code using large language models," *IEEE Access*, 2025.
- [13] G. Antal, L. Értekes, N. Szolnoki, and P. Hegedűs, "Evaluating retrieval-augmented generation for llm-based vulnerability detection: An empirical study on real-world java vulnerabilities," *IEEE Access*, 2026.
- [14] P.-C. Huang, C.-H. Mao, and C.-M. Lai, "Aegisguard: A context-aware framework for semantic vulnerability detection and risk stratification," *IEEE Access*, 2026.
- [15] H. Gao, D. Zhang, and L. Chen, "Scout: Llm-driven vulnerability detection via interactive repository exploration," *IEEE Access*, 2026.
- [16] I. S. Göçmen, A. S. Cezayir, and E. Tüzün, "Enhanced code reviews using pull request based change impact analysis," *Empirical Software Engineering*, vol. 30, no. 3, p. 64, 2025.
- [17] Z. Wei, J. Sun, Y. Sun, Y. Liu, D. Wu, Z. Zhang, X. Zhang, M. Li, Y. Liu, C. Li, M. Wan, J. Dong, and L. Zhu, "Advanced smart contract vulnerability detection via llm-powered multi-agent systems," *IEEE Transactions on Software Engineering*, 2025.
- [18] Y. Li, K. Joshi, X. Wang, and E. Wong, "Mavul: Multi-agent vulnerability detection via contextual reasoning and interactive refinement," in *2025 IEEE 7th International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA)*, 2025.
- [19] K. P. Reddy, J. S. Dhatteval, et al., "Agentic artificial intelligence with knowledge-based reasoning for automated vulnerability triage in collaborative software repositories," in *2026 IEEE 6th International Conference on Computing, Power, and Communication Technologies (IC2PCT)*. IEEE, 2026, pp. 493–498.
- [20] T. A. Syed, M. Riyaz Belgaum, S. Jan, A. A. Khan, and S. S. Alqahtani, "Agentic ai for autonomous defense in software supply chain security: Beyond provenance to vulnerability mitigation," in *2025 International Conference on Computer and Applications (ICCA)*, 2025.
- [21] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 1321–1330.
- [22] Semgrep, Inc., "Semgrep: Lightweight static analysis for many languages," <https://semgrep.dev>, accessed: Sep. 2026.
- [23] MITRE Corporation, "Common weakness enumeration (CWE)," <https://cwe.mitre.org>, accessed: Sep. 2026.
- [24] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. F. Bastien, "Bringing the web up to speed with WebAssembly," in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2017, pp. 185–200.
- [25] R. Wahbe, S. Lucco, T. E. Anderson, and S. L. Graham, "Efficient software-based fault isolation," in *Proceedings of the 14th ACM Symposium on Operating Systems Principles (SOSP)*, 1993, pp. 203–216.
- [26] G. W. Brier, "Verification of forecasts expressed in terms of probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.